



The University of Texas at Austin

Sept. 23, 2026

A Performance Portable Matrix Free Dense MTTKRP in GenTen

IEEE Cluster 2026

Gabriel Kosmacher¹, Eric T. Phipps², Sivasankaran Rajamanickam²

¹Oden Institute, The University of Texas at Austin

²Sandia National Laboratories



Predictive
Engineering &
Computational Science



Introduction: What is a tensor?

Vector

Order-1

$$\mathbf{x} \in \mathbb{R}^4$$

0
1
2
3

$$i = 0, \dots, 3$$

Row-major storage

0	1	2	3
---	---	---	---

Increasing address $\rightarrow$

$$\text{offset}(i) = i$$

Matrix

Order-2

$$A \in \mathbb{R}^{2 \times 3}$$

$j \rightarrow$	0	1	2
$i \downarrow$	3	4	5

Rows stored consecutively

Row-major storage

0	1	2	3	4	5
---	---	---	---	---	---

Increasing address $\rightarrow$

$$\text{offset}(i, j) = 3i + j$$

3-way tensor

Order-3

$$\mathcal{X} \in \mathbb{R}^{2 \times 2 \times 2}$$

$j \rightarrow$	0	2
$i \downarrow$	4	6
$k = 1$	1	3
	5	7

k : depth and fastest storage index

Row-major storage

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Increasing address $\rightarrow$

$$\text{offset}(i, j, k) = 4i + 2j + k$$

Introduction: What is a tensor?

Vector

Order-1

$$\mathbf{x} \in \mathbb{R}^4$$

0
1
2
3

$$i = 0, \dots, 3$$

Row-major storage

0	1	2	3
---	---	---	---

Increasing address

$$\text{offset}(i) = i$$

Matrix

Order-2

$$A \in \mathbb{R}^{2 \times 3}$$

$j \rightarrow$	0	1	2
$i \downarrow$	3	4	5

Rows stored consecutively

Row-major storage

0	1	2	3	4	5
---	---	---	---	---	---

Increasing address

$$\text{offset}(i, j) = 3i + j$$

3-way tensor

Order-3

$$\mathcal{X} \in \mathbb{R}^{2 \times 2 \times 2}$$

$j \rightarrow$	0	2
$i \downarrow$	4	6
$k = 1$	1	3
	5	7

k : depth and fastest storage index

Row-major storage

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Increasing address

$$\text{offset}(i, j, k) = 4i + 2j + k$$

Introduction: What is a tensor?

Vector

Order-1

$$\mathbf{x} \in \mathbb{R}^4$$

0
1
2
3

$$i = 0, \dots, 3$$

Row-major storage

0	1	2	3
---	---	---	---

Increasing address

$$\text{offset}(i) = i$$

Matrix

Order-2

$$A \in \mathbb{R}^{2 \times 3}$$

$j \rightarrow$

0	1	2
3	4	5

$i \downarrow$

Rows stored consecutively

Row-major storage

0	1	2	3	4	5
---	---	---	---	---	---

Increasing address

$$\text{offset}(i, j) = 3i + j$$

3-way tensor

Order-3

$$\mathcal{X} \in \mathbb{R}^{2 \times 2 \times 2}$$

$j \rightarrow$

0	2
4	6

$i \downarrow$

$k = 1$

k : depth and fastest storage index

Row-major storage

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Increasing address

$$\text{offset}(i, j, k) = 4i + 2j + k$$

Introduction: What is a tensor?

Vector

Order-1

$$\mathbf{x} \in \mathbb{R}^4$$

0
1
2
3

$$i = 0, \dots, 3$$

Row-major storage

0	1	2	3
---	---	---	---

Increasing address

$$\text{offset}(i) = i$$

Matrix

Order-2

$$A \in \mathbb{R}^{2 \times 3}$$

$j \rightarrow$	0	1	2
$i \downarrow$	3	4	5

Rows stored consecutively

Row-major storage

0	1	2	3	4	5
---	---	---	---	---	---

Increasing address

$$\text{offset}(i, j) = 3i + j$$

3-way tensor

Order-3

$$\mathcal{X} \in \mathbb{R}^{2 \times 2 \times 2}$$

$j \rightarrow$	0	2
$i \downarrow$	4	6
$k = 1$	5	7

k : depth and fastest storage index

Row-major storage

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

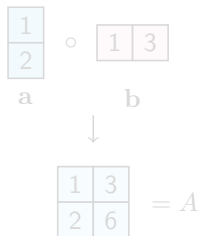
Increasing address

$$\text{offset}(i, j, k) = 4i + 2j + k$$

Introduction: Rank-one tensors

Rank-one matrix

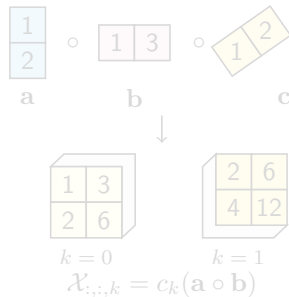
$$\mathbf{a} \in \mathbb{R}^I, \quad \mathbf{b} \in \mathbb{R}^J$$
$$A = \mathbf{a} \circ \mathbf{b} = \mathbf{a} \mathbf{b}^\top, \quad A_{ij} = a_i b_j$$



Every row is a multiple of $\mathbf{b}^\top$

Rank-one 3-way tensor

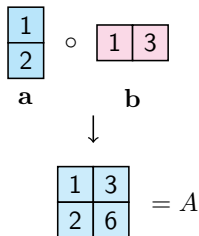
$$\mathbf{a} \in \mathbb{R}^I, \quad \mathbf{b} \in \mathbb{R}^J, \quad \mathbf{c} \in \mathbb{R}^K$$
$$\mathcal{X} = \mathbf{a} \circ \mathbf{b} \circ \mathbf{c}, \quad \mathcal{X}_{ijk} = a_i b_j c_k$$



Introduction: Rank-one tensors

Rank-one matrix

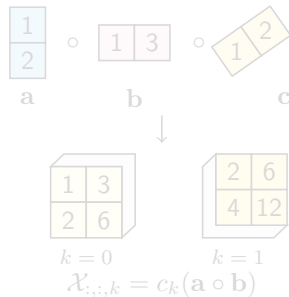
$$\mathbf{a} \in \mathbb{R}^I, \quad \mathbf{b} \in \mathbb{R}^J$$
$$A = \mathbf{a} \circ \mathbf{b} = \mathbf{a} \mathbf{b}^\top, \quad A_{ij} = a_i b_j$$



Every row is a multiple of $\mathbf{b}^\top$

Rank-one 3-way tensor

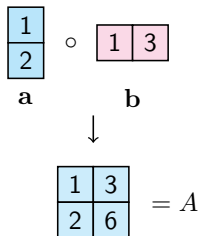
$$\mathbf{a} \in \mathbb{R}^I, \quad \mathbf{b} \in \mathbb{R}^J, \quad \mathbf{c} \in \mathbb{R}^K$$
$$\mathcal{X} = \mathbf{a} \circ \mathbf{b} \circ \mathbf{c}, \quad \mathcal{X}_{ijk} = a_i b_j c_k$$



Introduction: Rank-one tensors

Rank-one matrix

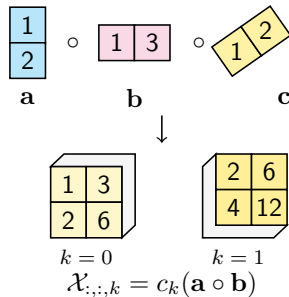
$$\mathbf{a} \in \mathbb{R}^I, \quad \mathbf{b} \in \mathbb{R}^J$$
$$A = \mathbf{a} \circ \mathbf{b} = \mathbf{a} \mathbf{b}^\top, \quad A_{ij} = a_i b_j$$



Every row is a multiple of $\mathbf{b}^\top$

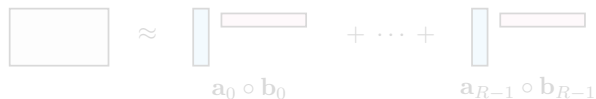
Rank-one 3-way tensor

$$\mathbf{a} \in \mathbb{R}^I, \quad \mathbf{b} \in \mathbb{R}^J, \quad \mathbf{c} \in \mathbb{R}^K$$
$$\mathcal{X} = \mathbf{a} \circ \mathbf{b} \circ \mathbf{c}, \quad \mathcal{X}_{ijk} = a_i b_j c_k$$



Introduction: Tensor compression

Matrix: sum of R rank-one terms


$$\approx \mathbf{a}_0 \circ \mathbf{b}_0 + \dots + \mathbf{a}_{R-1} \circ \mathbf{b}_{R-1}$$

$$M = AB^T = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r, \quad M_{ij} = \sum_{r=0}^{R-1} A_{ir} B_{jr}$$

Tensor: sum of R rank-one terms


$$\approx \mathbf{a}_0 \circ \mathbf{b}_0 \circ \mathbf{c}_0 + \dots + \mathbf{a}_{R-1} \circ \mathbf{b}_{R-1} \circ \mathbf{c}_{R-1}$$

$$\mathcal{M} = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r, \quad \mathcal{M}_{ijk} = \sum_{r=0}^{R-1} A_{ir} B_{jr} C_{kr}$$

Best rank- R approximation

$$\min_{A,B} \frac{1}{2} \|X - AB^T\|_F^2$$

Truncated SVD gives a global optimum: retain the R largest singular values and their vectors.

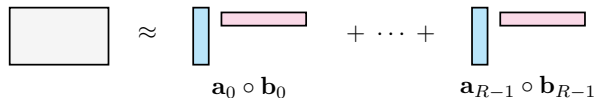
CP approximation

$$\min_{A,B,C} \frac{1}{2} \|\mathcal{X} - \mathcal{M}\|_F^2$$

No general closed-form solution. Iterative CP optimization uses gradients with tensor contractions (MTTKRP).

Introduction: Tensor compression

Matrix: sum of R rank-one terms


$$\mathbf{M} \approx \mathbf{a}_0 \circ \mathbf{b}_0 + \dots + \mathbf{a}_{R-1} \circ \mathbf{b}_{R-1}$$

$$\mathbf{M} = \mathbf{A}\mathbf{B}^\top = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r, \quad M_{ij} = \sum_{r=0}^{R-1} A_{ir} B_{jr}$$

Tensor: sum of R rank-one terms


$$\mathcal{M} \approx \mathbf{a}_0 \circ \mathbf{b}_0 \circ \mathbf{c}_0 + \dots + \mathbf{a}_{R-1} \circ \mathbf{b}_{R-1} \circ \mathbf{c}_{R-1}$$

$$\mathcal{M} = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r, \quad \mathcal{M}_{ijk} = \sum_{r=0}^{R-1} A_{ir} B_{jr} C_{kr}$$

Best rank- R approximation

$$\min_{\mathbf{A}, \mathbf{B}} \frac{1}{2} \|\mathbf{X} - \mathbf{A}\mathbf{B}^\top\|_F^2$$

Truncated SVD gives a global optimum: retain the R largest singular values and their vectors.

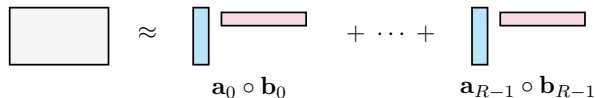
CP approximation

$$\min_{\mathbf{A}, \mathbf{B}, \mathbf{C}} \frac{1}{2} \|\mathcal{X} - \mathcal{M}\|_F^2$$

No general closed-form solution. Iterative CP optimization uses gradients with tensor contractions (MTTKRP).

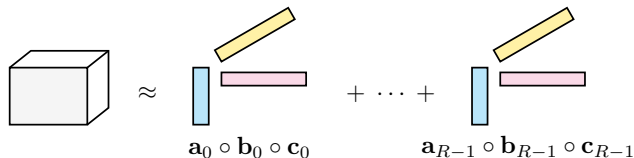
Introduction: Tensor compression

Matrix: sum of R rank-one terms


$$\approx \mathbf{a}_0 \circ \mathbf{b}_0 + \dots + \mathbf{a}_{R-1} \circ \mathbf{b}_{R-1}$$

$$M = AB^T = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r, \quad M_{ij} = \sum_{r=0}^{R-1} A_{ir} B_{jr}$$

Tensor: sum of R rank-one terms


$$\approx \mathbf{a}_0 \circ \mathbf{b}_0 \circ \mathbf{c}_0 + \dots + \mathbf{a}_{R-1} \circ \mathbf{b}_{R-1} \circ \mathbf{c}_{R-1}$$

$$\mathcal{M} = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r, \quad \mathcal{M}_{ijk} = \sum_{r=0}^{R-1} A_{ir} B_{jr} C_{kr}$$

Best rank- R approximation

$$\min_{A,B} \frac{1}{2} \|X - AB^T\|_F^2$$

Truncated SVD gives a global optimum: retain the R largest singular values and their vectors.

CP approximation

$$\min_{A,B,C} \frac{1}{2} \|\mathcal{X} - \mathcal{M}\|_F^2$$

No general closed-form solution. Iterative CP optimization uses gradients with tensor contractions (MTTKRP).

Introduction: CP as a matrix least-squares problem

Goal: choose A, B, C so their rank- R CP tensor approximates $\mathcal{X}$.

$$\min_{A,B,C} \frac{1}{2} \|\mathcal{X} - \mathcal{M}\|_F^2, \quad \mathcal{M} = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r.$$

CP-ALS: optimize one factor at a time, holding the other two fixed.

Update $A \longrightarrow$ update $B \longrightarrow$ update $C \longrightarrow$ repeat.

Each factor update is a matrix least-squares solve

$$B \leftarrow \arg \min_{\tilde{B}} \frac{1}{2} \|X_{(1)} - \tilde{B} Z_1^T\|_F^2.$$

⊙: columnwise Kronecker (Khatri–Rao) product. Weights are absorbed into the factors.

Introduction: CP as a matrix least-squares problem

Goal: choose A, B, C so their rank- R CP tensor approximates $\mathcal{X}$.

$$\min_{A,B,C} \frac{1}{2} \|\mathcal{X} - \mathcal{M}\|_F^2, \quad \mathcal{M} = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r.$$

CP-ALS: optimize one factor at a time, holding the other two fixed.

Update $A \longrightarrow$ update $B \longrightarrow$ update $C \longrightarrow$ repeat.

Mode-1 unfolding: turn the tensor into a matrix.

Keep j as the row index; combine (i, k) into the column index.

$$\mathcal{X} \in \mathbb{R}^{I \times J \times K} \longrightarrow X_{(1)} \in \mathbb{R}^{J \times IK}, \quad X_{(1)}[j, i + Ik] = X_{ijk}.$$

Each row contains one fixed- j slice; columns run through i first, then k . The entries are unchanged, so the Frobenius norm is preserved.

Each factor update is a matrix least-squares solve

$$B \leftarrow \arg \min_{\tilde{B}} \frac{1}{2} \|X_{(1)} - \tilde{B} Z_1^T\|_F^2.$$

⊙: columnwise Kronecker (Khatri–Rao) product. Weights are absorbed into the factors.

Introduction: CP as a matrix least-squares problem

Goal: choose A, B, C so their rank- R CP tensor approximates $\mathcal{X}$.

$$\min_{A,B,C} \frac{1}{2} \|\mathcal{X} - \mathcal{M}\|_F^2, \quad \mathcal{M} = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r.$$

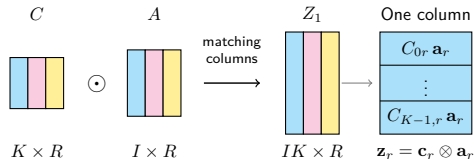
CP-ALS: optimize one factor at a time, holding the other two fixed.

Update $A \longrightarrow$ update $B \longrightarrow$ update $C \longrightarrow$ repeat.

Example: update B . Fix A, C :

$$M_{(1)} = BZ_1^T, \quad Z_1 = C \odot A.$$

The fixed factors form Z_1 ; only B is unknown.



Each factor update is a matrix least-squares solve

$$B \leftarrow \arg \min_{\tilde{B}} \frac{1}{2} \|X_{(1)} - \tilde{B} Z_1^T\|_F^2.$$

$\odot$: columnwise Kronecker (Khatri–Rao) product. Weights are absorbed into the factors.

Introduction: CP as a matrix least-squares problem

Goal: choose A, B, C so their rank- R CP tensor approximates $\mathcal{X}$.

$$\min_{A,B,C} \frac{1}{2} \|\mathcal{X} - \mathcal{M}\|_F^2, \quad \mathcal{M} = \sum_{r=0}^{R-1} \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r.$$

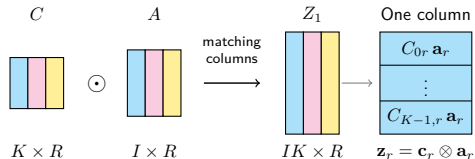
CP-ALS: optimize one factor at a time, holding the other two fixed.

Update $A \longrightarrow$ update $B \longrightarrow$ update $C \longrightarrow$ repeat.

Example: update B . Fix A, C :

$$M_{(1)} = BZ_1^T, \quad Z_1 = C \odot A.$$

The fixed factors form Z_1 ; only B is unknown.



Each factor update is a matrix least-squares solve

$$B \leftarrow \arg \min_{\tilde{B}} \frac{1}{2} \|X_{(1)} - \tilde{B}Z_1^T\|_F^2.$$

$\odot$: columnwise Kronecker (Khatri–Rao) product. Weights are absorbed into the factors.

Introduction: The MTTKRP kernel

Differentiate the matrix least-squares objective:

$$\nabla_B f = (BZ_1^T - X_{(1)})Z_1 = B(\underbrace{Z_1^T Z_1}_{\text{MTTKRP}}) - \underbrace{X_{(1)}Z_1}_{\text{MTTKRP}}.$$

Matricized tensor times Khatri–Rao product:

$$\begin{array}{ccc} G^{(1)} & = & X_{(1)} \times Z_1 = C \odot A \\ \text{Mode-1 output} & & \text{Unfolded tensor} \quad \text{Khatri-Rao product} \end{array}$$

One kernel for gradients and alternating least squares

For gradients, contract the residual: $\nabla_B f = (M_{(1)} - X_{(1)})Z_1$.

For ALS, solve $B(Z_1^T Z_1) = G^{(1)}$, with $G^{(1)} = X_{(1)}Z_1$.

Introduction: The MTTKRP kernel

Differentiate the matrix least-squares objective:

$$\nabla_B f = (BZ_1^T - X_{(1)})Z_1 = B(\underbrace{Z_1^T Z_1}_{\text{MTTKRP}}) - \underbrace{X_{(1)}Z_1}_{\text{MTTKRP}}.$$

Matricized tensor times Khatri–Rao product:

$$\begin{array}{ccc} G^{(1)} & & X_{(1)} \\ \text{Mode-1 output} & = & \text{Unfolded tensor} \end{array} \times \begin{array}{c} Z_1 = C \odot A \\ \text{Khatri–Rao product} \end{array}$$

$J \times R$ $J \times IK$ $IK \times R$

One kernel for gradients and alternating least squares

For gradients, contract the residual: $\nabla_B f = (M_{(1)} - X_{(1)})Z_1$.

For ALS, solve $B(Z_1^T Z_1) = G^{(1)}$, with $G^{(1)} = X_{(1)}Z_1$.

Introduction: The MTTKRP kernel

Differentiate the matrix least-squares objective:

$$\nabla_B f = (BZ_1^T - X_{(1)})Z_1 = B(\underbrace{Z_1^T Z_1}_{\text{MTTKRP}}) - \underbrace{X_{(1)}Z_1}_{\text{MTTKRP}}.$$

Matricized tensor times Khatri–Rao product:

$$\begin{array}{ccccc} G^{(1)} & & X_{(1)} & & Z_1 = C \odot A \\ \text{Mode-1 output} & = & \text{Unfolded tensor} & \times & \text{Khatri–Rao product} \end{array}$$

One kernel for gradients and alternating least squares

For gradients, contract the residual: $\nabla_B f = (M_{(1)} - X_{(1)})Z_1$.

For ALS, solve $B(Z_1^T Z_1) = G^{(1)}$, with $G^{(1)} = X_{(1)}Z_1$.

Introduction: Mode-1 MTTKRP complications

$$G^{(1)} = X_{(1)} Z_1 \quad Z_1 = C \odot A$$

Unfolding reorders the tensor

Tensor storage: k varies fastest

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Unfolded storage: group by j

0	4	1	5	2	6	3	7
---	---	---	---	---	---	---	---

$j = 0$

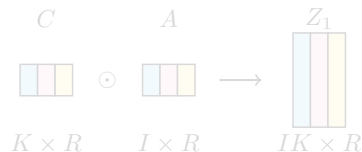
$j = 1$

Numbers are original memory offsets; $I = J = K = 2$.

Mode-1 rows fix j , but the tensor's row-major storage interleaves those rows.

Explicit unfolding moves $\mathcal{O}(IJK)$ entries.

The Khatri–Rao product is redundant



Materialize $Z_1 = C \odot A$

$Z_1 \in \mathbb{R}^{IK \times R}$, RIK stored scalars

Store only the two factors

$A \in \mathbb{R}^{I \times R}$, $C \in \mathbb{R}^{K \times R}$
 $R(I + K)$ stored scalars

Introduction: Mode-1 MTTKRP complications

$$G^{(1)} = X_{(1)} Z_1 \quad Z_1 = C \odot A$$

Unfolding reorders the tensor

Tensor storage: k varies fastest

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---



Unfolded storage: group by j

0	4	1	5	2	6	3	7
---	---	---	---	---	---	---	---

$j = 0$

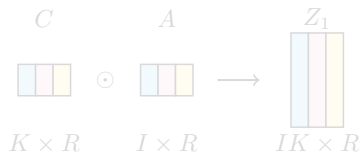
$j = 1$

Numbers are original memory offsets; $I = J = K = 2$.

Mode-1 rows fix j , but the tensor's row-major storage interleaves those rows.

Explicit unfolding moves $\mathcal{O}(IJK)$ entries.

The Khatri–Rao product is redundant



Materialize $Z_1 = C \odot A$

$Z_1 \in \mathbb{R}^{IK \times R}$, RIK stored scalars

Store only the two factors

$A \in \mathbb{R}^{I \times R}$, $C \in \mathbb{R}^{K \times R}$
 $R(I + K)$ stored scalars

Introduction: Mode-1 MTTKRP complications

$$G^{(1)} = X_{(1)} Z_1 \quad Z_1 = C \odot A$$

Unfolding reorders the tensor

Tensor storage: k varies fastest

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---



Unfolded storage: group by j

0	4	1	5	2	6	3	7
---	---	---	---	---	---	---	---

$j = 0$

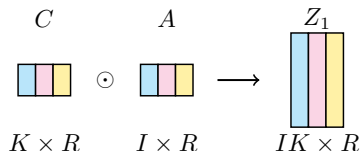
$j = 1$

Numbers are original memory offsets; $I = J = K = 2$.

Mode-1 rows fix j , but the tensor's row-major storage interleaves those rows.

Explicit unfolding moves $\mathcal{O}(IJK)$ entries.

The Khatri–Rao product is redundant



Materialize $Z_1 = C \odot A$

$Z_1 \in \mathbb{R}^{IK \times R}$, RIK stored scalars

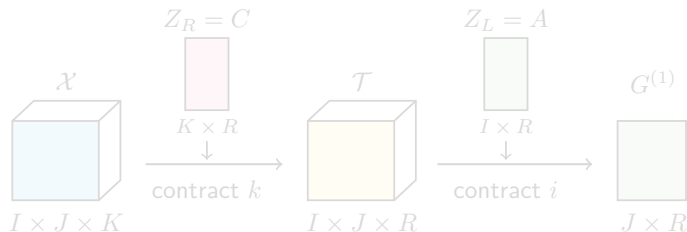
Store only the two factors

$A \in \mathbb{R}^{I \times R}$, $C \in \mathbb{R}^{K \times R}$

$R(I + K)$ stored scalars

Introduction: MTTKRP-GEMM

Phan et al. (2013) [1]: split the contraction into two steps



$$T_{ijr} = \sum_{k=0}^{K-1} X_{ijk} C_{kr}, \quad G_{jr}^{(1)} = \sum_{i=0}^{I-1} A_{ir} T_{ijr}$$

No explicit unfolding: $T = \text{reshape}(\mathcal{X}, [IJ, K]) C$ uses a row-major view.

No full Z_1 : combine the right-contraction intermediate with the left factor.

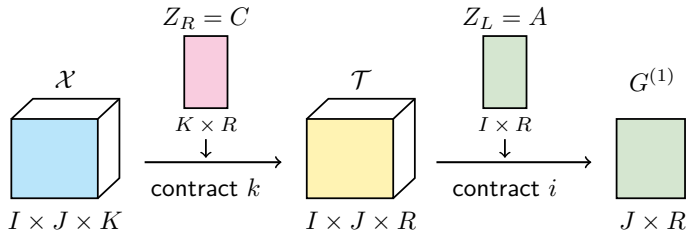
Endpoint modes still need large Khatri–Rao products!

At modes 0 and $d - 1$, one side is empty; the other is the full Z .

$$Z_0 = C \odot B : \boxed{RJK} \text{ scalars}, \quad Z_2 = B \odot A : \boxed{RIJ} \text{ scalars}.$$

Introduction: MTTKRP-GEMM

Phan et al. (2013) [1]: split the contraction into two steps



$$T_{ijr} = \sum_{k=0}^{K-1} X_{ijk} C_{kr}, \quad G_{jr}^{(1)} = \sum_{i=0}^{I-1} A_{ir} T_{ijr}$$

No explicit unfolding: $T = \text{reshape}(\mathcal{X}, [IJ, K]) C$ uses a row-major view.

No full Z_1 : combine the right-contraction intermediate with the left factor.

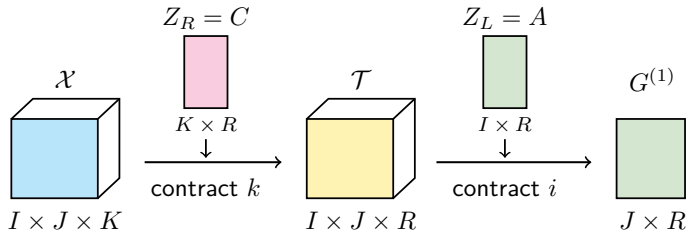
Endpoint modes still need large Khatri–Rao products!

At modes 0 and $d - 1$, one side is empty; the other is the full Z .

$$Z_0 = C \odot B : \boxed{RJK} \text{ scalars}, \quad Z_2 = B \odot A : \boxed{RIJ} \text{ scalars}.$$

Introduction: MTTKRP-GEMM

Phan et al. (2013) [1]: split the contraction into two steps



$$T_{ijr} = \sum_{k=0}^{K-1} X_{ijk} C_{kr}, \quad G_{jr}^{(1)} = \sum_{i=0}^{I-1} A_{ir} T_{ijr}$$

No explicit unfolding: $T = \text{reshape}(\mathcal{X}, [IJ, K]) C$ uses a row-major view.

No full Z_1 : combine the right-contraction intermediate with the left factor.

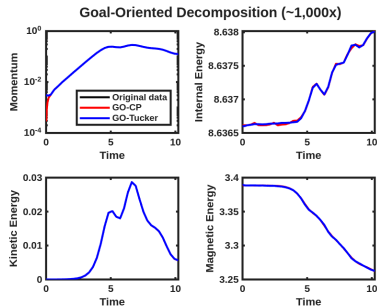
Endpoint modes still need large Khatri–Rao products!

At modes 0 and $d - 1$, one side is empty; the other is the full Z .

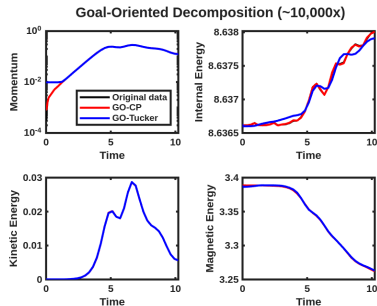
$$Z_0 = C \odot B : \boxed{RJK} \text{ scalars}, \quad Z_2 = B \odot A : \boxed{RIJ} \text{ scalars}.$$

Introduction: Why high CP rank?

3D island coalescence: goal-oriented CP and Tucker [2]



CP rank $R = 2294$; $\approx 1000\times$ compression
CP field error: 1.1% unscaled / 5.3% scaled.
Reproduced from Fig. 9(b), Dunlavy et al. [2].



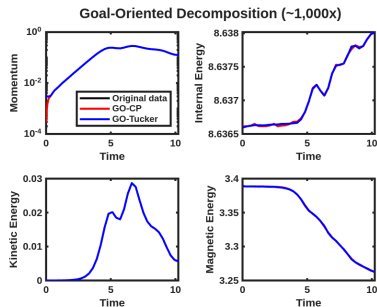
CP rank $R = 229$; $\approx 10,000\times$ compression
CP field error: 2.4% unscaled / 18% scaled.
Reproduced from Fig. 9(d), Dunlavy et al. [2].

Qols can be preserved at lower rank; accurate fields need more CP terms.

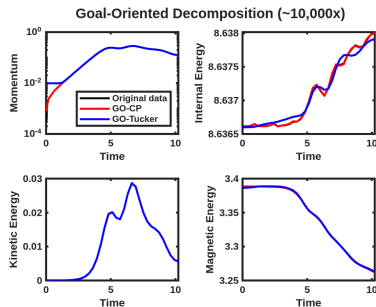
Qols: quantities of interest. Field errors are relative Frobenius errors.

Introduction: Why high CP rank?

3D island coalescence: goal-oriented CP and Tucker [2]



CP rank $R = 2294$; $\approx 1000\times$ compression
CP field error: 1.1% unscaled / 5.3% scaled.
Reproduced from Fig. 9(b), Dunlavy et al. [2].



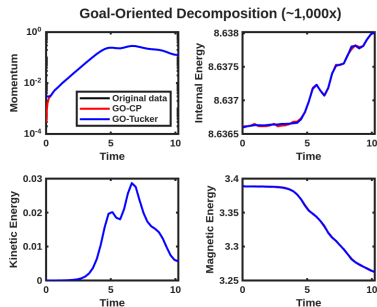
CP rank $R = 229$; $\approx 10,000\times$ compression
CP field error: 2.4% unscaled / 18% scaled.
Reproduced from Fig. 9(d), Dunlavy et al. [2].

Qols can be preserved at lower rank; accurate fields need more CP terms.

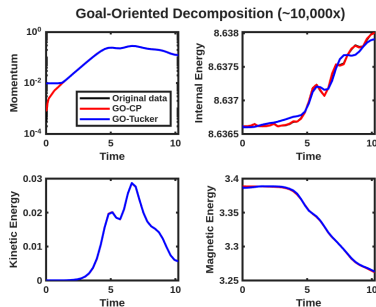
Qols: quantities of interest. Field errors are relative Frobenius errors.

Introduction: Why high CP rank?

3D island coalescence: goal-oriented CP and Tucker [2]



CP rank $R = 2294$; $\approx 1000\times$ compression
CP field error: 1.1% unscaled / 5.3% scaled.
Reproduced from Fig. 9(b), Dunlavy et al. [2].



CP rank $R = 229$; $\approx 10,000\times$ compression
CP field error: 2.4% unscaled / 18% scaled.
Reproduced from Fig. 9(d), Dunlavy et al. [2].

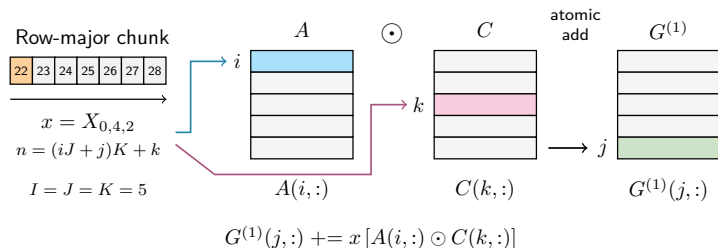
Qols can be preserved at lower rank; accurate fields need more CP terms.

Qols: quantities of interest. Field errors are relative Frobenius errors.

Methods: MTTKRP-ELEM

MTTKRP-ELEM: an element-wise formulation commonly used for sparse tensors [3].

$$G_{jr}^{(1)} = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} A_{ir} C_{kr}, \quad 0 \leq j < J, \quad 0 \leq r < R.$$



Parallelize over tensor entries: gather factor rows, multiply across ranks, and accumulate into the output row.

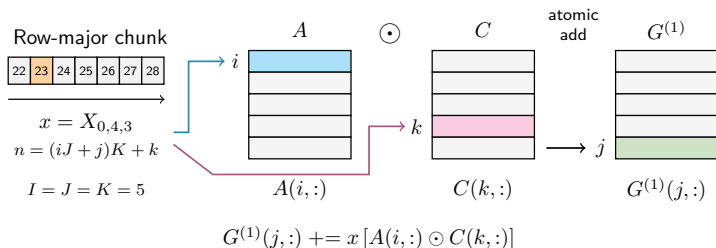
Tradeoff: no unfolding or explicit Z_1 , but limited row reuse and contended atomic updates.

Here $\odot$ denotes the Hadamard (element-wise) product; initialize $G^{(1)} = 0$.

Methods: MTTKRP-ELEM

MTTKRP-ELEM: an element-wise formulation commonly used for sparse tensors [3].

$$G_{jr}^{(1)} = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} A_{ir} C_{kr}, \quad 0 \leq j < J, \quad 0 \leq r < R.$$



Parallelize over tensor entries: gather factor rows, multiply across ranks, and accumulate into the output row.

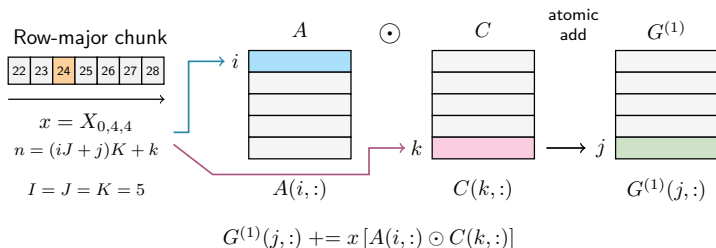
Tradeoff: no unfolding or explicit Z_1 , but limited row reuse and contended atomic updates.

Here $\odot$ denotes the Hadamard (element-wise) product; initialize $G^{(1)} = 0$.

Methods: MTTKRP-ELEM

MTTKRP-ELEM: an element-wise formulation commonly used for sparse tensors [3].

$$G_{jr}^{(1)} = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} A_{ir} C_{kr}, \quad 0 \leq j < J, \quad 0 \leq r < R.$$



Parallelize over tensor entries: gather factor rows, multiply across ranks, and accumulate into the output row.

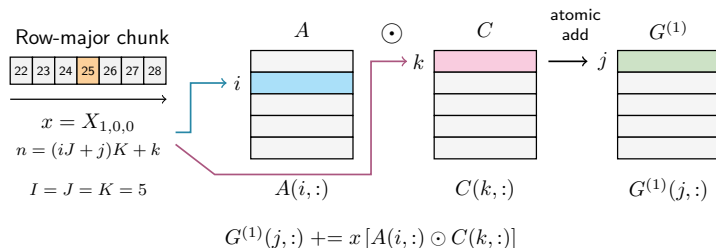
Tradeoff: no unfolding or explicit Z_1 , but limited row reuse and contended atomic updates.

Here $\odot$ denotes the Hadamard (element-wise) product; initialize $G^{(1)} = 0$.

Methods: MTTKRP-ELEM

MTTKRP-ELEM: an element-wise formulation commonly used for sparse tensors [3].

$$G_{jr}^{(1)} = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} A_{ir} C_{kr}, \quad 0 \leq j < J, \quad 0 \leq r < R.$$



Parallelize over tensor entries: gather factor rows, multiply across ranks, and accumulate into the output row.

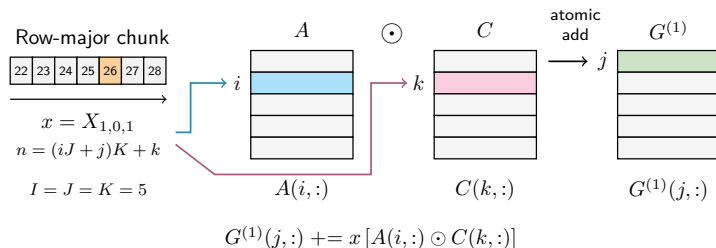
Tradeoff: no unfolding or explicit Z_1 , but limited row reuse and contended atomic updates.

Here $\odot$ denotes the Hadamard (element-wise) product; initialize $G^{(1)} = 0$.

Methods: MTTKRP-ELEM

MTTKRP-ELEM: an element-wise formulation commonly used for sparse tensors [3].

$$G_{jr}^{(1)} = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} A_{ir} C_{kr}, \quad 0 \leq j < J, \quad 0 \leq r < R.$$



Parallelize over tensor entries: gather factor rows, multiply across ranks, and accumulate into the output row.

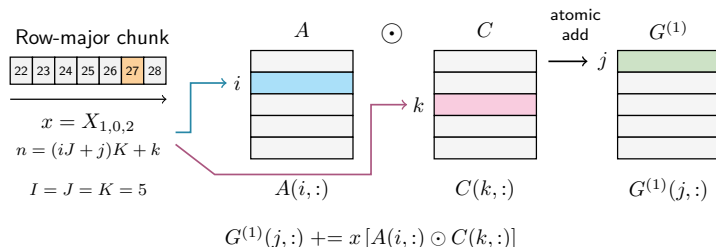
Tradeoff: no unfolding or explicit Z_1 , but limited row reuse and contended atomic updates.

Here $\odot$ denotes the Hadamard (element-wise) product; initialize $G^{(1)} = 0$.

Methods: MTTKRP-ELEM

MTTKRP-ELEM: an element-wise formulation commonly used for sparse tensors [3].

$$G_{jr}^{(1)} = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} A_{ir} C_{kr}, \quad 0 \leq j < J, \quad 0 \leq r < R.$$



Parallelize over tensor entries: gather factor rows, multiply across ranks, and accumulate into the output row.

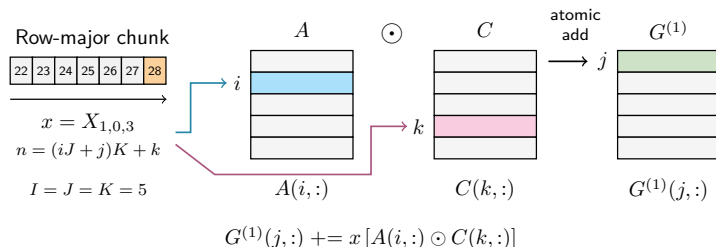
Tradeoff: no unfolding or explicit Z_1 , but limited row reuse and contended atomic updates.

Here $\odot$ denotes the Hadamard (element-wise) product; initialize $G^{(1)} = 0$.

Methods: MTTKRP-ELEM

MTTKRP-ELEM: an element-wise formulation commonly used for sparse tensors [3].

$$G_{jr}^{(1)} = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} A_{ir} C_{kr}, \quad 0 \leq j < J, \quad 0 \leq r < R.$$



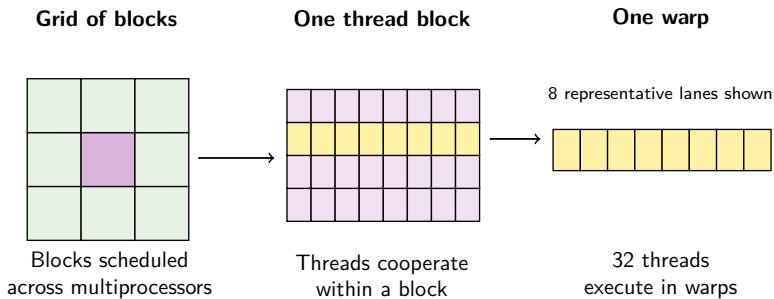
Parallelize over tensor entries: gather factor rows, multiply across ranks, and accumulate into the output row.

Tradeoff: no unfolding or explicit Z_1 , but limited row reuse and contended atomic updates.

Here $\odot$ denotes the Hadamard (element-wise) product; initialize $G^{(1)} = 0$.

Aside: GPU parallelization

A kernel launches a **grid of thread blocks**; each block contains **warps**.



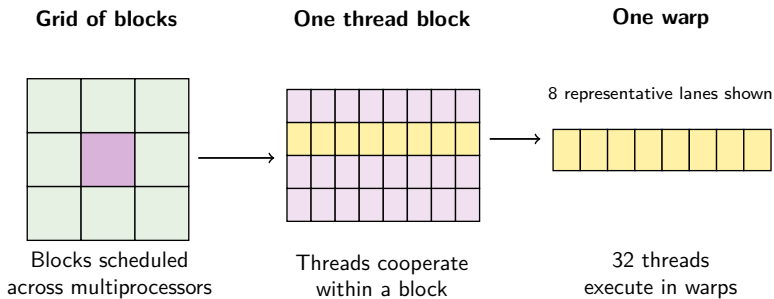
Connect the hierarchy to MTTKRP

Assign tensor work to blocks and threads; distribute rank columns across lanes. Threads accumulate partial results in registers.

Schematic CUDA hierarchy. Blocks need not run simultaneously; threads in a block can synchronize and share on-chip memory.

Aside: GPU parallelization

A kernel launches a **grid of thread blocks**; each block contains **warps**.



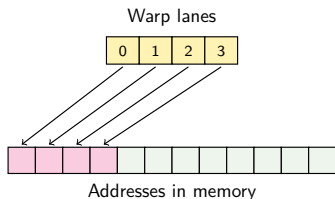
Connect the hierarchy to MTTKRP

Assign tensor work to blocks and threads; distribute rank columns across lanes. Threads accumulate partial results in registers.

Schematic CUDA hierarchy. Blocks need not run simultaneously; threads in a block can synchronize and share on-chip memory.

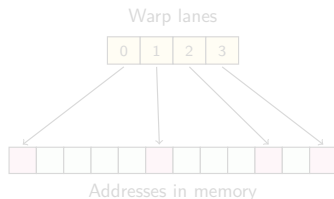
Aside: memory access matters

Coalesced accesses



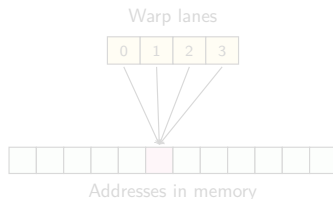
Nearby addresses can be served by fewer memory transactions.

Scattered accesses



Spread-out addresses require more transactions, wasting bandwidth.

Contended atomics



Updates to the same address must be ordered; contention limits throughput.

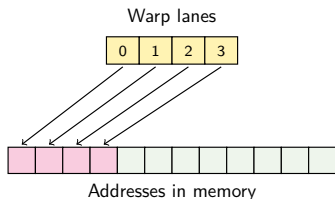
Why group tensor entries?

ELEM repeatedly reads factor rows and atomically updates $G^{(1)}$. Can we do better?

Four representative lanes shown. Scattered accesses need more transactions, not necessarily serial execution.

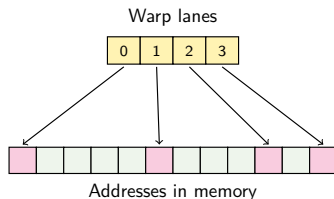
Aside: memory access matters

Coalesced accesses



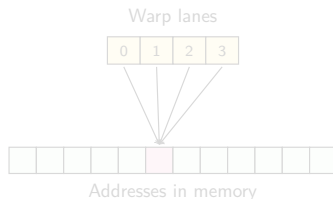
Nearby addresses can be served by fewer memory transactions.

Scattered accesses



Spread-out addresses require more transactions, wasting bandwidth.

Contended atomics



Updates to the same address must be ordered; contention limits throughput.

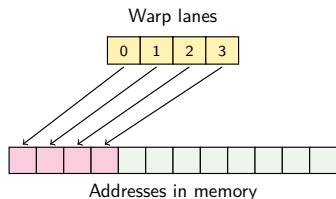
Why group tensor entries?

ELEM repeatedly reads factor rows and atomically updates $G^{(1)}$. Can we do better?

Four representative lanes shown. Scattered accesses need more transactions, not necessarily serial execution.

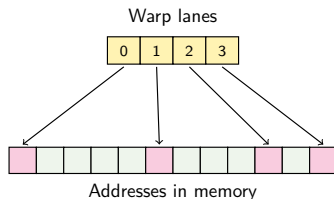
Aside: memory access matters

Coalesced accesses



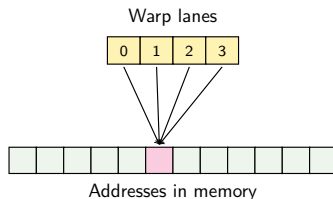
Nearby addresses can be served by fewer memory transactions.

Scattered accesses



Spread-out addresses require more transactions, wasting bandwidth.

Contended atomics



Updates to the same address must be ordered; contention limits throughput.

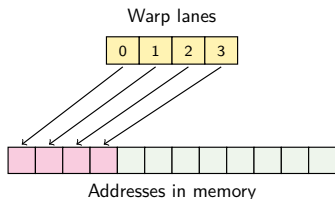
Why group tensor entries?

ELEM repeatedly reads factor rows and atomically updates $G^{(1)}$. Can we do better?

Four representative lanes shown. Scattered accesses need more transactions, not necessarily serial execution.

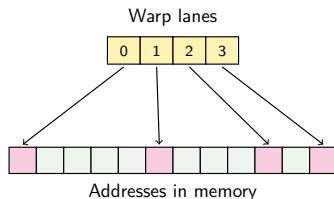
Aside: memory access matters

Coalesced accesses



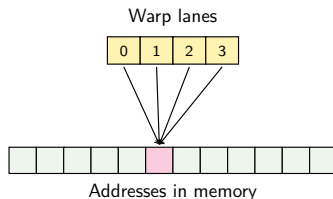
Nearby addresses can be served by fewer memory transactions.

Scattered accesses



Spread-out addresses require more transactions, wasting bandwidth.

Contended atomics



Updates to the same address must be ordered; contention limits throughput.

Why group tensor entries?

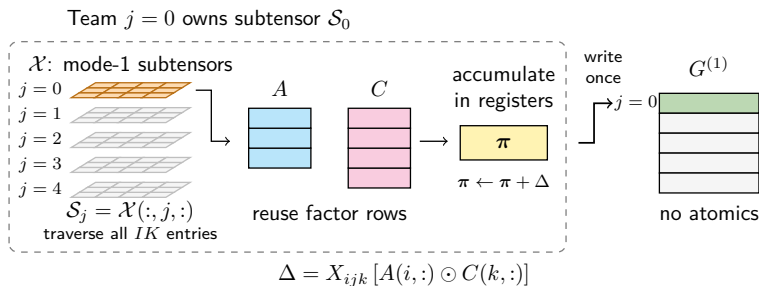
ELEM repeatedly reads factor rows and atomically updates $G^{(1)}$. Can we do better?

Four representative lanes shown. Scattered accesses need more transactions, not necessarily serial execution.

Methods: MTTKRP-SUB

Group work by output row: one team processes the entire subtensor $\mathcal{S}_j = \mathcal{X}(:, j, :)$.

$$G^{(1)}(j, :) = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} [A(i, :) \odot C(k, :)].$$



Benefits: reuse factor data; accumulate locally; write without atomics.

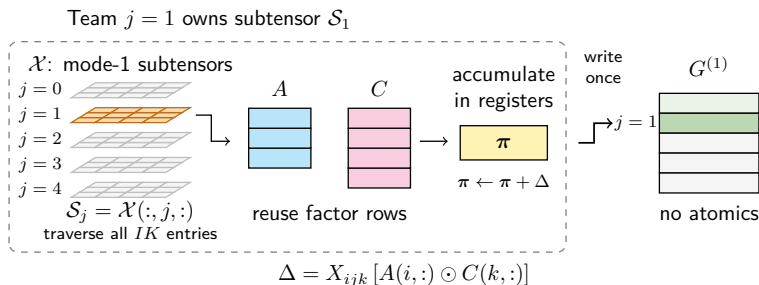
Tradeoff: only J independent subtenors, each with IK entries to traverse.

$N_T = IK$ (the full subtensor); π starts at zero. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-SUB

Group work by output row: one team processes the entire subtensor $\mathcal{S}_j = \mathcal{X}(:, j, :)$.

$$G^{(1)}(j, :) = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} [A(i, :) \odot C(k, :)].$$



Benefits: reuse factor data; accumulate locally; write without atomics.

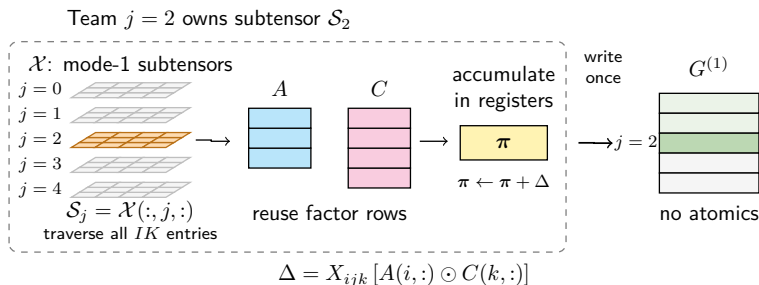
Tradeoff: only J independent subtenors, each with IK entries to traverse.

$N_T = IK$ (the full subtensor); π starts at zero. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-SUB

Group work by output row: one team processes the entire subtensor $\mathcal{S}_j = \mathcal{X}(:, j, :)$.

$$G^{(1)}(j, :) = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} [A(i, :) \odot C(k, :)].$$



Benefits: reuse factor data; accumulate locally; write without atomics.

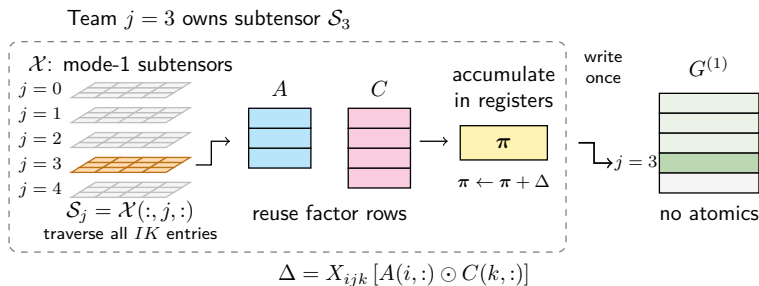
Tradeoff: only J independent subtensors, each with IK entries to traverse.

$N_T = IK$ (the full subtensor); π starts at zero. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-SUB

Group work by output row: one team processes the entire subtensor $\mathcal{S}_j = \mathcal{X}(:, j, :)$.

$$G^{(1)}(j, :) = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} [A(i, :) \odot C(k, :)].$$



Benefits: reuse factor data; accumulate locally; write without atomics.

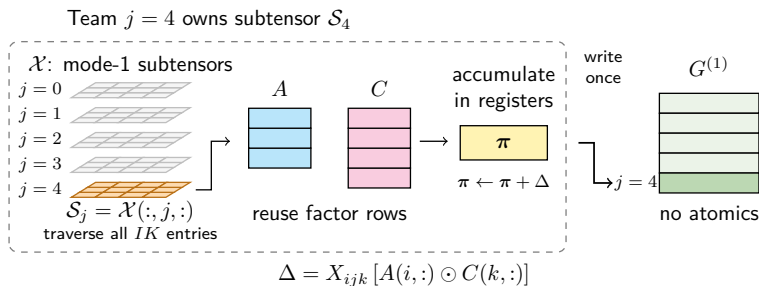
Tradeoff: only J independent subtensors, each with IK entries to traverse.

$N_T = IK$ (the full subtensor); π starts at zero. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-SUB

Group work by output row: one team processes the entire subtensor $\mathcal{S}_j = \mathcal{X}(:, j, :)$.

$$G^{(1)}(j, :) = \sum_{i=0}^{I-1} \sum_{k=0}^{K-1} X_{ijk} [A(i, :) \odot C(k, :)].$$



Benefits: reuse factor data; accumulate locally; write without atomics.

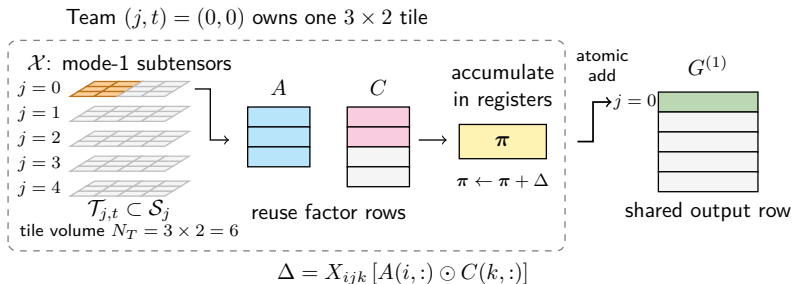
Tradeoff: only J independent subtensors, each with IK entries to traverse.

$N_T = IK$ (the full subtensor); π starts at zero. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

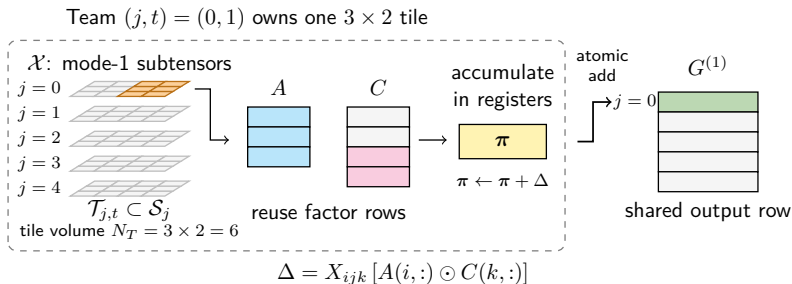
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3, J = 5, K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

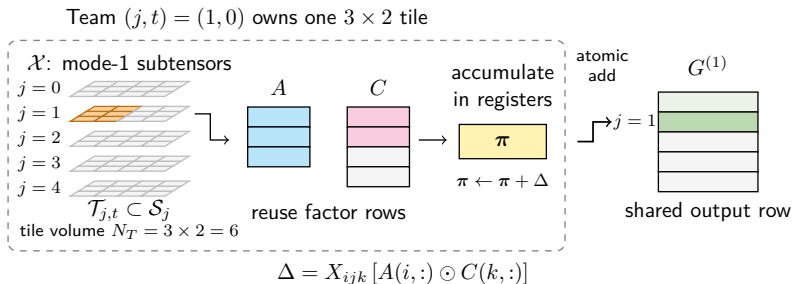
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

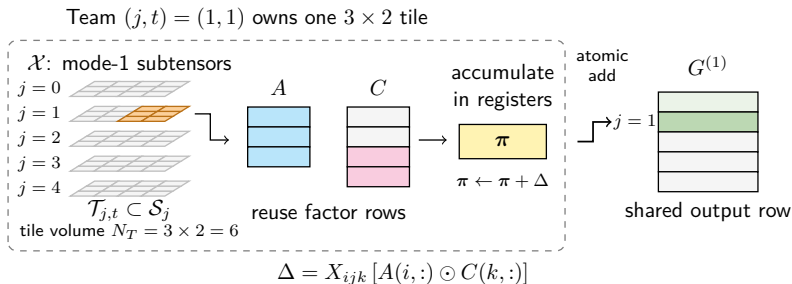
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

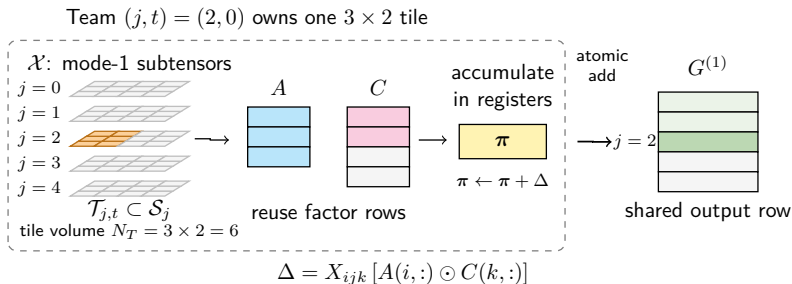
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

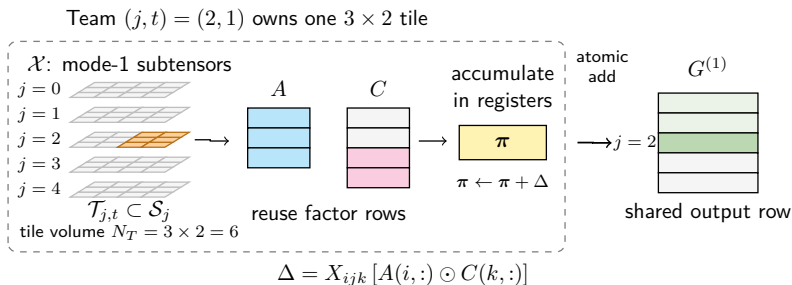
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

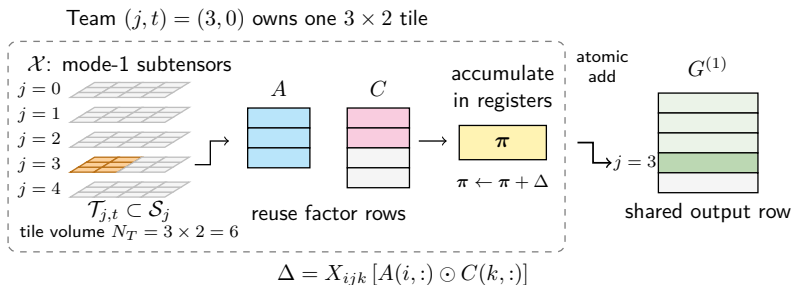
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

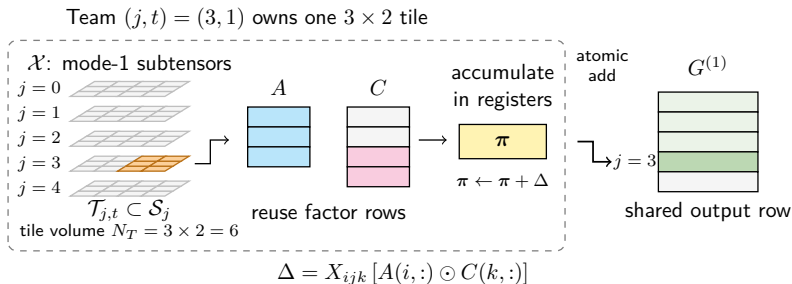
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

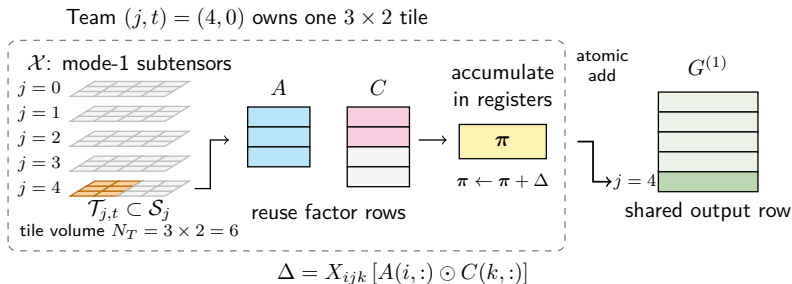
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



Balance: factor reuse within a tile, with more independent teams than SUB.

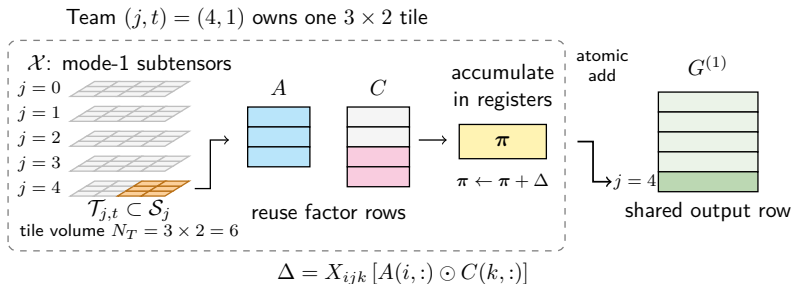
Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Methods: MTTKRP-TILE

Split each subtensor into tiles: one team processes N_T entries, then adds a partial output row.

$$\pi_{j,t} = \sum_{(i,k) \in \mathcal{T}_{j,t}} X_{ijk} [A(i,:) \odot C(k,:)], \quad G^{(1)}(j,:) = \sum_t \pi_{j,t}.$$



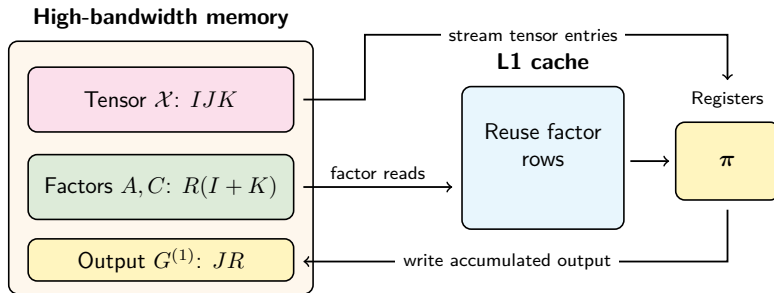
Balance: factor reuse within a tile, with more independent teams than SUB.

Fewer atomics than ELEM: one update per tile per rank, rather than per entry per rank.

Example: $I = 3$, $J = 5$, $K = 4$; two 3×2 tiles per subtensor. Here $\odot$ denotes the Hadamard product.

Performance Analysis: a simplified memory model

For $G^{(1)}$, stream $\mathcal{X}$ and repeatedly access rows of A and C .



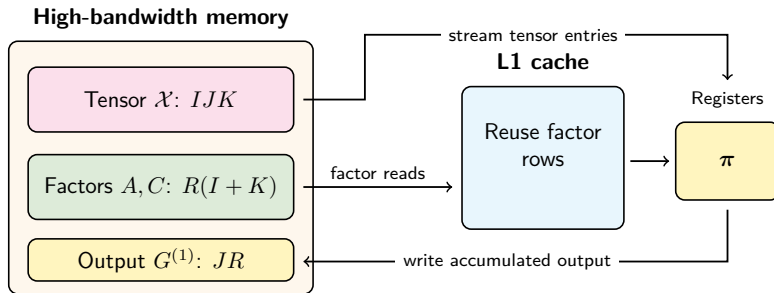
The opportunity: serve repeated factor reads from cache.

The constraint: cache capacity and bandwidth are both finite.

Sizes shown in scalars; multiply by s_f for bytes. Simplified GPU view: the model uses L1 on GPUs and L2 on CPUs (paper, Sec. IV-B).

Performance Analysis: a simplified memory model

For $G^{(1)}$, stream $\mathcal{X}$ and repeatedly access rows of A and C .



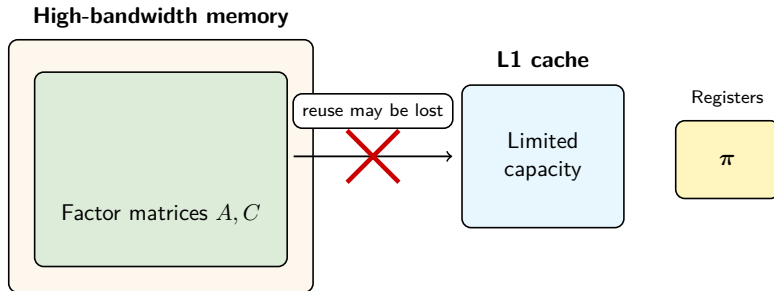
The opportunity: serve repeated factor reads from cache.

The constraint: cache capacity and bandwidth are both finite.

Sizes shown in scalars; multiply by s_f for bytes. Simplified GPU view: the model uses L1 on GPUs and L2 on CPUs (paper, Sec. IV-B).

Performance Analysis: make factor reuse fit

MTTKRP-SUB: a full $I \times K$ slice touches all rows of A and C .



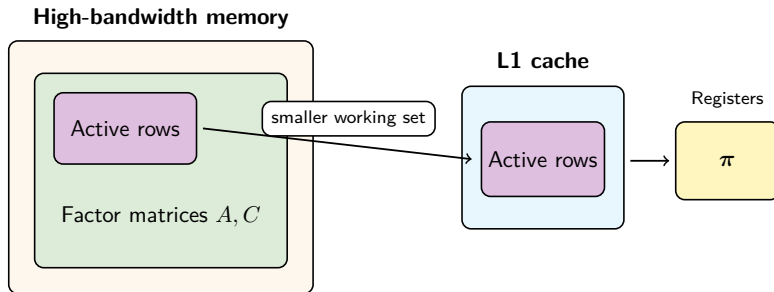
Active factor data: $s_f R_b (I + K)$ bytes.

Large working sets can evict factor rows before they are reused.

R_b : rank columns processed together ($R_b \leq R$). Factor working sets only; cache is also shared by other active work.

Performance Analysis: make factor reuse fit

MTTKRP-TILE: a $T_I \times T_K$ tile touches fewer factor rows.



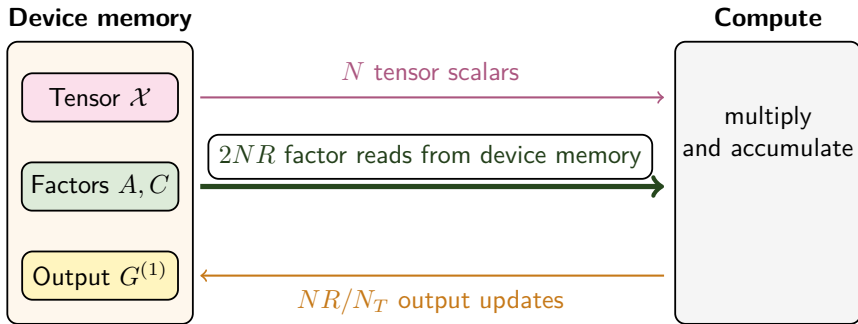
Active factor data: $s_f R_b (T_I + T_K)$ bytes.

Smaller tiles improve the opportunity for cache reuse, but require more output updates.

R_b : rank columns processed together ($R_b \leq R$). Factor working sets only; cache is also shared by other active work.

Performance Analysis: three traffic models

1. **No cache reuse:** every factor read pays the device-memory cost.



$$m_0 = s_f(N + 2NR + NR/N_T).$$

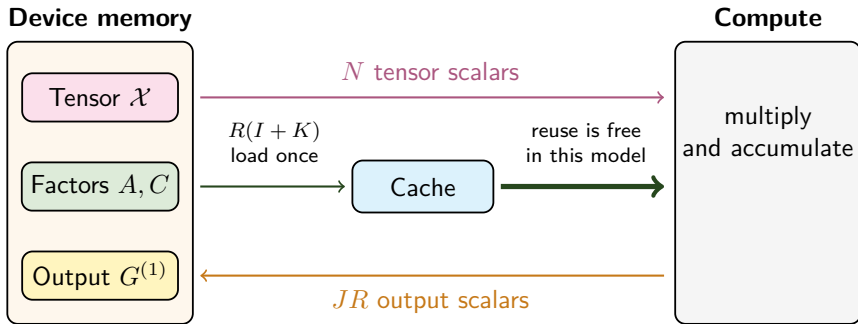
Takeaway: repeated factor reads can dominate traffic.

$N = IJK$; N_T : entries per tile; s_f : bytes per scalar. $T_q = 3NR/\tau_f + m_q/\tau_m$. Paper, Sec. IV-B.

$m_{0,LM}$ is bandwidth-weighted traffic, not storage size. Weight-vector and transaction overhead are omitted.

Performance Analysis: three traffic models

2. **Infinite cache:** load data once; subsequent reuse is free.



$$m_{\infty} = s_f(N + R(I + K) + JR).$$

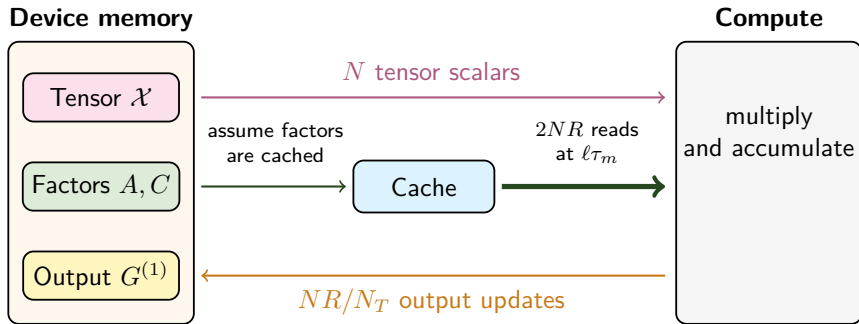
Takeaway: an optimistic baseline that ignores the cost of cache access.

$N = IJK$; N_T : entries per tile; s_f : bytes per scalar. $T_q = 3NR/\tau_f + m_q/\tau_m$. Paper, Sec. IV-B.

$m_{0,LM}$ is bandwidth-weighted traffic, not storage size. Weight-vector and transaction overhead are omitted.

Performance Analysis: three traffic models

3. Finite cache bandwidth: reuse is cheaper, but not free.



$$m_{0,\text{LM}} = s_f(N + NR/N_T + 2NR/\ell), \quad \ell = \tau_{\text{LM}}/\tau_m.$$

Takeaway: model the benefit of reuse without making it free.

$N = IJK$; N_T : entries per tile; s_f : bytes per scalar. $T_q = 3NR/\tau_f + m_q/\tau_m$. Paper, Sec. IV-B.

$m_{0,\text{LM}}$ is bandwidth-weighted traffic, not storage size. Weight-vector and transaction overhead are omitted.

Results: assessing the memory models

Measured time T versus model predictions (seconds); CP rank $R = 32$.

$\mathcal{A}$: tearing mode (4-way); $\mathcal{B}$: island coalescence (5-way).

		OpenMP (CPU)				CUDA (H100)				HIP (MI300A)			
		T	T_0	$T_{0,LM}$	T_∞	T	T_0	$T_{0,LM}$	T_∞	T	T_0	$T_{0,LM}$	T_∞
$\mathcal{A}$	ELEM	10.394	3.817	1.349	0.056	1.823	0.138	0.047	0.003	2.668	0.087	0.028	0.002
	SUB	0.543	2.877	0.409	0.056	8.528	0.104	0.013	0.003	16.267	0.066	0.007	0.002
	TILE	0.526	2.955	0.487	0.056	0.043	0.110	0.019	0.003	0.063	0.069	0.011	0.002
$\mathcal{B}$	ELEM	25.167	9.876	3.054	0.130	3.924	0.356	0.105	0.007	7.995	0.225	0.063	0.004
	SUB	1.137	7.927	1.105	0.130	23.032	0.286	0.035	0.007	47.401	0.181	0.019	0.004
	TILE	1.517	8.414	1.592	0.130	0.120	0.304	0.052	0.007	0.202	0.188	0.026	0.004

Gray: measured T . Boxes: selected models from the paper's interpretation, not nearest-value fits.

Predicted time = compute time + memory time: $T_q = NRd/\tau_f + m_q/\tau_m$.

Divide the flop count by peak compute throughput, then add modeled bytes divided by memory bandwidth.

Takeaway: cache reuse matters for CPU SUB and TILE; GPU predictions remain optimistic.

Paper, Table III and Sec. IV-B. d : tensor order; $m_{0,LM}$ weights cache traffic by its bandwidth. TILE uses the paper's tile-size heuristic.

Results: memory cost changes the comparison

Island coalescence: $\mathcal{B} \in \mathbb{R}^{129 \times 129 \times 129 \times 12 \times 39}$.

Memory growth

Dashed lines: 80 GB per H100.

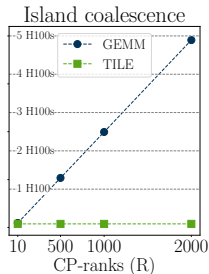


Fig. 3: peak storage over all modes.

TILE wins at $R = 10, 500$ on one H100; GEMM pays for redistribution.

At $R = 2000$, GEMM is faster, but needs **six H100s across two nodes**; TILE still fits on one.

Paper, Sec. IV-D; CP-ALS tolerance 10^{-4} . Capacity lines are lower bounds on GPU counts.

One GPU versus multiple GPUs

CP-ALS time; lower is better.

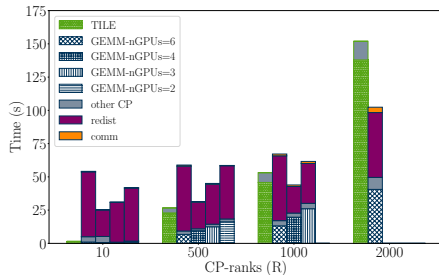


Fig. 6: includes redistribution and communication.

Future work: toward the compute roofline

Increase arithmetic intensity: reuse more data per byte moved.

Current performance

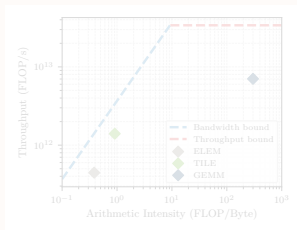


Fig. 4: tearing mode, $R = 2000$, H100.

Next: more reuse without forming Z

- 2D output tiling with shared-memory caching.
- Thread and warp tiling.
- Auto-tune league and team sizes.
- Hilbert-curve or ALTO-like tensor ordering for better locality.

Goal: move TILE toward the compute-bound regime while preserving its low memory footprint.

Paper, Fig. 4 and Sec. V. Roofline intensity uses m_{∞} for GEMM and $m_{0,LM}$ for ELEM/TILE.

Future work: toward the compute roofline

Increase arithmetic intensity: reuse more data per byte moved.

Current performance

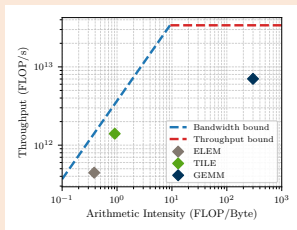


Fig. 4: tearing mode, $R = 2000$, H100.

Next: more reuse without forming Z

- 2D output tiling with shared-memory caching.
- Thread and warp tiling.
- Auto-tune league and team sizes.
- Hilbert-curve or ALTO-like tensor ordering for better locality.

Goal: move TILE toward the compute-bound regime while preserving its low memory footprint.

Paper, Fig. 4 and Sec. V. Roofline intensity uses m_{∞} for GEMM and $m_{0,LM}$ for ELEM/TILE.

Future work: toward the compute roofline

Increase arithmetic intensity: reuse more data per byte moved.

Current performance

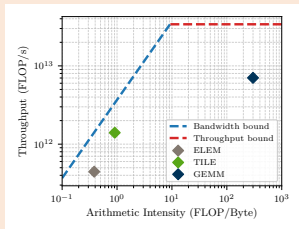


Fig. 4: tearing mode, $R = 2000$, H100.

Next: more reuse without forming Z

- 2D output tiling with shared-memory caching.
- Thread and warp tiling.
- Auto-tune league and team sizes.
- Hilbert-curve or ALTO-like tensor ordering for better locality.

Goal: move TILE toward the compute-bound regime while preserving its low memory footprint.

Paper, Fig. 4 and Sec. V. Roofline intensity uses m_{∞} for GEMM and $m_{0,LM}$ for ELEM/TILE.

- [1] A.-H. Phan, P. Tichavský, and A. Cichocki, “Fast alternating LS algorithms for high order CANDECOMP/PARAFAC tensor factorizations,” *IEEE Transactions on Signal Processing*, vol. 61, no. 19, pp. 4834–4846, 2013. DOI: 10.1109/TSP.2013.2269903
- [2] D. M. Dunlavy, E. T. Phipps, H. Kolla, J. N. Shadid, and E. Phillips, *Goal-oriented low-rank tensor decompositions for numerical simulation data*, Accepted for publication in SIAM Journal on Scientific Computing; journal version not yet available online, 2025. arXiv: 2508.11139 [math.NA]. [Online]. Available: <https://arxiv.org/abs/2508.11139>
- [3] E. T. Phipps and T. G. Kolda, “Software for sparse tensor decomposition on emerging computing architectures,” *SIAM Journal on Scientific Computing*, vol. 41, no. 3, pp. C269–C290, 2019. DOI: 10.1137/18M1210691 eprint: <https://doi.org/10.1137/18M1210691>. [Online]. Available: <https://doi.org/10.1137/18M1210691>

Thank you!

Questions?

Gabriel Kosmacher

Eric T. Phipps Sivasankaran Rajamanickam



Predictive
Engineering &
Computational Science



Sandia
National
Laboratories

Takeaway: use GEMM when its intermediates fit;
use TILE to reduce memory requirements and avoid distribution.

Available now in GenTen: our matrix-free MTTKRP implementations.

github.com/sandialabs/GenTen

Acknowledgments

This work was partially supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, under contract DE-NA0003525.